Information and Software Technology 187 (2025) 107854

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

An empirical study on the accuracy of GitHub's dependency graph and the nature of its inaccuracy

Daniele Bifolco [a,*], Simone Romano [b], Sabato Nocera [b], Rita Francese [b], Giuseppe Scanniello [b], Massimiliano Di Penta [a]

[a] University of Sannio, Via Traiano, 9, Benevento, 82100, Italy
[b] University of Salerno, Via Giovanni Paolo II, 132, Fisciano, 84084, Italy

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.15412179

Keywords:
GitHub
Dependency graph
Empirical study

ABSTRACT

Context: GitHub's dependency graph is a tool that eases Software Composition Analysis (SCA), and it is leveraged not only by other tools or by practitioners in their analyses but also by researchers when conducting studies on open-source projects. However, its potential inaccuracy may seriously harm its applicability and usefulness.

Objective: This paper quantitatively and qualitatively analyzes the accuracy of GitHub's dependency graphs for Java and Python projects, how such accuracy has changed over time, and what the likely pitfalls and limitations of the dependency graph are.

Method: After creating statistically significant samples of Java and Python projects, we analyzed their dependency graph in two directions, forward (by looking at dependencies), backward (by looking at dependents), and inspected their manifest/lock files.

Results: Results indicate that in our sample, dependencies have over 27% of inaccuracy, and dependents up to 10%. Errors depend on several reasons, among others, an oversimplified processing of manifest/lock files by the dependency graph generator.

Conclusion: Our results provide (i) guidelines for researchers to understand the threats arising in studies based on the dependency graph and (ii) insights to practitioners and tool builders to enhance their SCA, given the current limitations of the dependency graph.

1. Introduction

Software Composition Analysis (SCA) is a crucial activity in software development, and it entails the analysis of components that a software system requires at various stages of its development life cycle, *e.g.,* some components may be needed at run-time for the deployed system, while others may only be necessary during testing. Relevant applications of SCA include: identifying and tracing buggy or vulnerable components a software project depends upon; licensing compatibility analysis; or, in general, any analysis to check whether a software system depends on components that can create trustworthiness problems (*e.g.,* because being developed by non-trusted contributors or in hostile countries). SCA may also be relevant for researchers, *e.g.,* those interested in studying Machine Learning (ML) intensive projects may want to analyze software projects that depend on popular ML frameworks.

Let us consider the most popular forge (*i.e.,* a web-based hosting platform for open-source projects) for open-source projects [1], *i.e.,* GitHub. Since 2017, GitHub has provided a very relevant feature for SCA: the *dependency graph* [2]. GitHub's dependency graph shows, for those repositories for which it is enabled, the lists of *dependencies* and *dependents*, respectively. According to GitHub's terminology, dependencies are repositories and packages the repository depends on, while dependents are repositories that depend on that repository's packages. GitHub infers the dependency graph by analyzing manifest and lock files—*i.e.,* files, such as `pom.xml` for Maven or `requirements.txt` for pip, specifying project build dependencies. For this reason, GitHub's dependency graph is only available for projects written with programming languages for which package managers are supported (*e.g.,* the dependency graph is not available for C because none of its package managers are supported) [3]. Other than leveraging manifest/lock files, users can manually add dependencies to their project's dependency graph using the *dependency submission API*. The dependency graph is automatically updated when a commit changing a (supported) manifest/lock file to the default branch is pushed on GitHub or when anyone pushes a change to a dependent repository.

* Corresponding author.
E-mail address: d.bifolco@studenti.unisannio.it (D. Bifolco).

https://doi.org/10.1016/j.infsof.2025.107854
Received 15 May 2025; Received in revised form 20 July 2025; Accepted 21 July 2025
Available online 29 July 2025
0950-5849/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Practitioners and researchers have been leveraging GitHub's dependency graph. For instance, practitioners have been using tools that depend on GitHub's dependency graph. One of them is *Dependabot* [4], a GitHub tool to monitor security vulnerabilities in the dependencies of a given repository and keep the dependencies up-to-date—in 2022, Dependabot opened more than 75 million pull requests to update repositories' dependencies [5]. Another tool based on GitHub's dependency graph is the one that allows generating *SBOMs* (*Software Bill of Materials*),[1] released by GitHub in 2023. While most of the existing SBOM-generation tools leverage manifest/lock files directly, GitHub's one allows the generation of an SBOM directly from the dependency page of a repository without cloning the repository and using a third-party tool locally.

As far as the research is concerned, GitHub's dependency graph has been leveraged in several studies. On the one side, some studies indirectly relied on GitHub's dependency graph because they analyze SBOMs [6–9]. On the other side, some studies analyzed different types of ecosystems, *e.g.,* ML-intensive projects [10] or projects leveraging pre-trained models [11] by mining the dependents of given libraries through GitHub's dependency graph. The accuracy of the analyses conducted by the tools mentioned above (*i.e.,* Dependabot, the SBOM generator, or any other tool leveraging GitHub's dependency graph), as well as the findings of studies based on the dependency graph's data, strongly depends on the extent to which such information is accurate.

This paper reports an empirical study that quantitatively and qualitatively analyzes accuracy issues occurring in GitHub's dependency graph. The analysis has been conducted on statistically significant samples (with a confidence level of 95% and a margin of error of 5%) of Java and Python projects hosted on GitHub and satisfying given filtering criteria. Moreover, to investigate whether the dependency graph accuracy has changed over time, we repeated the sampling (and the analysis) at two different times, April 2023 and September 2024. The first snapshot consists of 296 Java and 338 Python projects, and the second of 292 Java and 327 Python projects.

We analyzed the dependency graph accuracy from two perspectives, complementing each other:

1. A *backward analysis*, focusing on the accuracy of the dependencies, as they are reported in GitHub's dependency graph of a given repository.
2. A *forward analysis*, focusing on the accuracy of the dependents.

On top of such analyses, we analyzed the correspondence between the dependencies reported in GitHub's dependency graph of a given repository and those reported in the corresponding manifest or lock files. Last but not least, we performed qualitative analysis on a statistically significant sample of dependency graph mismatches to identify possible error causes. This qualitative analysis allowed us to distill implications for different stakeholders, including practitioners, analysis tool creators (including GitHub itself), and researchers.

This paper extends our previous work [12] along different dimensions. First of all, we repeated the dependency graph analysis over time. Second, we conducted an extensive qualitative analysis of the dependency graph inaccuracies to identify the likely root causes and possible solutions. Last, we expanded the discussion of results and their practical implications.

The rest of the paper is organized as follows. Section 2 discusses related research that leveraged GitHub's dependency graph or dealt with analyzing software systems' interdependencies. Section 3, provides the study definition, research questions, and context. Then, Sections 4 and 5 present the quantitative and qualitative components of the study, respectively. Section 6 discusses the study's implications and possible threats to validity. Finally, Section 7 concludes the paper and outlines directions for future work.

[1] An SBOM is a formal and machine-readable inventory of the dependencies of a given software project, including versions, vendors, licenses, and other information on the dependencies.

2. Related work

To identify work related to ours, we conducted a quasi-systematic search on Google Scholar by using the query "GitHub dependency graph". Our search procedure, inspired by common practices in exploratory literature reviews [13], involved an initial plan to review the first ten pages of results and monitor for diminishing returns. However, the search returned only four pages. We reviewed these results and included all pertinent contributions as related work. These were then integrated with other contributions that we knew were relevant to our work.

To discuss the studies that leveraged GitHub's dependency graph and contrast them with the research presented in this paper, we focused on the following characteristics:

- **Year.** The publication year of the study.
- **Aim.** The objective pursued by the study (*e.g.,* analyzing tool adoption or developers' behavior).
- **Context.** An overview of the context of the study (*e.g.,* number of projects or pull requests analyzed).
- **Key Findings.** A summary of the main results emerging from the study.
- **Use of dependency graph.** How the study used GitHub's dependency graph (*e.g.,* directly to retrieve GitHub projects' dependencies or indirectly to generate SBOMs or through Dependabot).
- **Discussions on dependency graph.** Whether the study discusses the limitations of GitHub's dependency graph (*e.g.,* accuracy concerns).

Table 1 summarizes the aforementioned characteristics for the identified studies. Below, we discuss these past studies grouped by their use of GitHub's dependency graph.

A first group [6,7,14,20] focuses on *Dependabot and security updates*. Alfadel et al. [6] examined the degree to which developers adopt Dependabot for updating vulnerable dependencies. They observed that more than half of the security pull requests suggested by Dependabot were accepted and merged within a day. Mohayeji et al. [14] and He et al. [7] also performed an empirical study on the use of Dependabot's pull requests. Mohayeji et al. [14] focused on the effectiveness of Dependabot in maintaining the dependencies secure in JavaScript projects. He et al. [7] evaluated the effectiveness of Dependabot in dependency maintenance. The study revealed that using Dependabot reduced the technical lag, and developers were highly receptive to its pull requests. Ivanova et al. [20] evaluated four SCA tools, including Dependabot, using 13 Maven projects and 11 crafted attack scenarios. Although the aforementioned studies [6,7,14,20] indicate that Dependabot's automated pull requests are effective and well-accepted, none discussed any potential limitations of the underlying GitHub's dependency graph.

A second group uses GitHub's dependency graph to *retrieve projects' dependencies* [8,20]. To unveil the technical roles of developers on GitHub (*e.g.,* back-end, front-end), Montandon et al. [8] gathered 1000 projects' dependencies from GitHub's dependency graph. Ivanova et al. [20] leveraged GitHub's dependency graph to double-check the alerts of Dependabot. Both Montandon et al. [8] and Ivanova et al. [20] never mentioned potential inaccuracies in the dependencies retrieved through GitHub's dependency graph.

A third group uses GitHub's dependency graph to *retrieve projects' dependents* [9,15–19]. Nocera et al. [9] retrieved repositories dependent on SPDX and CycloneDX SBOM generators, observing a low but rising adoption of such generators. Toma and Bezemer [15] and Idowu et al. [19] retrieved repositories dependent on TensorFlow, PyTorch, or Scikit-learn. Toma and Bezemer [15] investigated how datasets and models are stored and versioned in open-source ML applications, while Idowu et al. [19] characterized the type and evolution

Table 1
Summary of related work leveraging GitHub's dependency graph.

Paper	Year	Aim	Context	Key findings	Use of dependency graph	Discussions on dependency graph
[6]	2021	Study the adoption of Dependabot for security updates	2904 JavaScript projects hosted on GitHub and 15,243 security pull requests	65.42% of security pull requests were merged within a day	Through Dependabot	No discussion of the limitations of GitHub's dependency graph
[8]	2021	Identify technical roles of developers on GitHub	2284 developers labeled with six different roles	ML classifiers can recommend roles for developers	To retrieve projects' dependencies	No discussion of the limitations of GitHub's dependency graph
[14]	2023	Effects of Dependabot in handling security updates	978 JavaScript projects hosted on GitHub and 4195 security pull requests	Developers accept most pull requests automatically generated by Dependabot	Through Dependabot	No discussion of the limitations of GitHub's dependency graph
[7]	2023	Effects of Dependabot in dependency maintenance	1823 projects hosted on GitHub, 502,752 pull requests, and 131 developers surveyed	Dependabot reduces technical lag and developers are receptive to its pull requests	Through Dependabot	No discussion of the limitations of GitHub's dependency graph
[9]	2023	Study the adoption of SBOM generators in GitHub projects	All projects from GitHub dependent on SBOM generators by SPDX or CycloneDX	Low adoption of SBOM generators but the trend is increasing	To retrieve projects' dependents	Found and discarded 11 false positives while verifying the retrieved dependents
[15]	2024	Study how datasets and models are stored/versioned in ML applications	93 projects from GitHub dependent on TensorFlow, PyTorch, or Scikit-learn	Most datasets and models are stored in the file system and lack version control integration	To retrieve projects' dependents	No discussion of the limitations of GitHub's dependency graph
[16]	2024	Analyze challenges in ML asset management	15,065 Question & Answers posts on Stack Overflow, tool specific forums, and GitHub/GitLab	Software environment and dependency are top pain points (18.89%)	To retrieve projects' dependents	No discussion of the limitations of GitHub's dependency graph
[17]	2024	Investigate API misuses in data-centric libraries beyond deep learning	345 Stack Overflow posts and 358 GitHub commits using five data-centric libraries	49 API misuses identified; 55% were data-dependent and 39% stem from data conversion errors	To retrieve projects' dependents	Removal of duplicates, *i.e.,* projects that were retrieved more than once
[18]	2024	Retrieve and validate repository information for PyPI packages	4,227,425 PyPI releases and 16,439 package-repository links	PyRadar achieves 90.2% repository retrieval accuracy with source-based techniques	To retrieve projects' dependents	Limitations related to missing or incorrect dependents and issues like package name collision. The authors manually validated the retrieved links.
[19]	2024	Characterize ML project types and evolution	31,066 projects hosted on GitHub dependent on TensorFlow or Scikit-learn	ML projects are research- and education-oriented	To retrieve projects' dependents	Found and discarded false positives while validating the retrieved dependents
[20]	2024	Evaluate the robustness of metadata-based SCA tools	13 Java Maven projects	All tools showed poor resilience against adversarial manipulations	Through Dependabot and to retrieve projects' dependencies	No discussion of the limitations of GitHub's dependency graph
[21]	2024	Evaluate the correctness of four SBOM generators, including the one based on the dependency graph	7876 projects from GitHub	All SBOM generators, including the one based on the dependency graph, generated inaccurate SBOMs	Through GitHub's SBOM generator	Highlighted inaccuracies of the generated SBOMs, without focusing on GitHub's dependency graph
This work	2025	Quantitatively and qualitatively analyze the accuracy of GitHub's dependency graph	298 Java Maven and 340 Python pip/Poetry projects	Most projects, for a variety of causes, show inaccuracies in the dependencies and dependents of GitHub's dependency graph	To retrieve projects' dependencies and dependents	The goal of the work is to discuss the pitfalls and limitations of GitHub's dependency graph

of such repositories. Similarly, Zhao et al. [16] retrieved repositories relevant to ML applications and asset management tools. Galappaththi et al. [17] categorized API misuses of repositories dependent on NumPy, pandas, and scikit-learn. Lastly, Gao et al. [18] proposed a framework to retrieve and validate repository information for PyPI releases. Among those studies, Toma and Bezemer [15] and Zhao et al. [16] were the only ones not discussing any limitation of GitHub's dependency graph in retrieving projects' dependents. Nocera et al. [9] and Idowu et al. [19] found and discarded false positives, *i.e.,* repositories that did not actually depend upon the target repositories. Galappaththi et al. [17] discarded duplicate projects' dependents. Gao et al. [18] manually validated the dependents of the retrieved projects to account for potential omissions, inaccuracies, and issues such as package name collisions. While raising questions about the accuracy of GitHub's dependency graph, these studies [9,17–19] did not investigate such accuracy concerns.

The fourth and final group, consisting of a single study [21], uses GitHub's dependency graph as *SBOM generator*. Yu et al. [21] assessed

SBOMs generated through GitHub's dependency graph and three other SBOM generators, namely Trivy, Syft, and Microsoft SBOM Tool. The analyzed SBOMs were generated from 7876 open-source projects across nine programming languages. The study results revealed that all SBOMs suffer from inaccuracies in the representation of dependencies, including those generated through GitHub's dependency graph. Among the causes of such inaccuracies, Yu et al. reported (i) incomplete metadata parsing, (ii) limitations in resolving transitive dependencies, (iii) limited support for version constraints, and (iv) inconsistent package naming conventions and dependency definitions. Our paper shares underlying motivations with that of Yu et al. [21] in assessing the accuracy of GitHub's dependency graph. However, the work by Yu et al. [21] fundamentally diverges from ours in both scope and methodology. In particular, Yu et al. [21] did not distinguish GitHub's dependency graph inaccuracies from those of the other tools, while we focus uniquely on GitHub's dependency graph. We conducted three analyses of the accuracy of GitHub's dependency graph, examining dependencies (backward analysis), dependents (forward analysis), and the correspondence with manifest/lock files (manifest/lock file analysis). We also repeated these three analyses over time to assess any change in the accuracy of GitHub's dependency graph. In addition, we perform a systematic, qualitative categorization of the types of inaccuracies occurring in the GitHub dependency graph and, whenever possible, their root causes.

In summary, our work contributes to the literature by identifying and characterizing key threats inherent GitHub's dependency graph. These findings offer valuable perspectives for researchers seeking to design more robust empirical studies, while also informing practitioners and tool developers aiming to enhance SCA in light of the current limitations of GitHub's dependency data.

3. Study definition and context

The **goal** of our study is to analyze, by using mining software repository techniques, GitHub's dependency graph with the **purpose** of (i) assessing its accuracy and (ii) identifying the likely causes behind inaccuracies (if any) in GitHub's dependency graph. The **perspective** is that of practitioners who rely on GitHub's dependency graph or any tool built upon it (*e.g.,* those for generating SBOMs or Dependabot) and researchers who leverage GitHub's dependency graph for their studies. The **context** consists of open-source projects hosted on GitHub written in Java and Python, using Maven and pip/Poetry as package managers, respectively.

3.1. Research questions

Based on the above-mentioned study goal, we formulated and then studied two Research Questions (RQs):

RQ1. *To what extent does GitHub's dependency graph provide accurate information?*

This RQ assesses the accuracy of both dependencies and dependents provided by GitHub's dependency graph. From the researchers' perspective, an incorrect dependency graph may produce inaccurate results in their study. From the practitioners' perspective, an inaccurate graph may result in incomplete or inaccurate SBOM, thus leading to possible security issues or licensing violations. To study RQ1, in September 2024, we replicated the analyses performed in our previous paper [12] on the accuracy of the dependencies and dependents provided by GitHub's dependency graph.

Longitudinal analysis. To understand if the accuracy of Github's dependency graph changed over time, we compared two snapshots:

- **First snapshot.** This is the snapshot of the analyses performed in our previous paper [12] on the accuracy of the dependencies and dependents provided by GitHub's dependency graph. This snapshot was taken in April 2023.
- **Second snapshot.** This is the last snapshot of the analyses on the accuracy of the dependencies and dependents provided by GitHub's dependency graph. As mentioned above, this snapshot was taken in September 2024.

The comparison mentioned above should allow for the discovery of whether the accuracy of GitHub's dependency graph has improved, regressed, or remained relatively stable over time.

RQ2. *What are the likely causes for the error cases occurring in GitHub's dependency graph?*

This RQ aims to investigate the error cases encountered during our analysis of GitHub's dependency graph, to deliver to practitioners and researchers the likely causes behind them. Given that practitioners and researchers widely rely on GitHub's dependency graph for different reasons (*e.g.,* monitoring security vulnerabilities in projects' dependencies with Dependabot or searching dependents/dependencies when conducting empirical studies), understanding the likely causes of these error cases may matter. By identifying the likely causes, we hope to provide insights into possible mitigations, improvements to GitHub's dependency graph, and best practices for practitioners to minimize errors in their dependency metadata.

3.2. Study context and planning

The context of our study, as in our initial work [12], consists of open-source software projects hosted on GitHub written in Java and Python, the former using Maven as a package manager, the latter employing pip or Poetry as a package manager. We focused on the Java and Python programming languages because of their popularity—according to *PYPL index*, Python is the most popular programming language, followed by Java [22]. The choice of the package managers is driven by the compatibility with GitHub's dependency graph. Maven is the only package manager supported for Java, while pip and Poetry are the only ones supported for Python [2].

To search for software projects on GitHub, we leveraged *GHS* [23]—a tool to facilitate the selection of projects hosted on GitHub when conducting mining software repository studies. While doing so, we kept in mind the perils of mining GitHub by Kalliamvakou et al. [24]. Therefore, with the support of GHS, we searched for projects that met the following inclusion criteria:

1. **Developed in Java or Python.** This was to focus on these two popular programming languages (as previously mentioned).
2. **Not archived and with at least one commit made in the month preceding the query date (April 2023).** This was to avoid selecting software projects that are inactive [24].
3. **With at least 100 stars, at least 100 commits, and at least one fork.** This was to mitigate the risk of selecting personal projects [24].
4. **Not fork.** This was to limit the risk of selecting duplicate projects [24].

The search with GHS returned 7172 software projects (2296 Java projects and 4876 Python projects) satisfying Criteria 1–4. Later, we applied an additional inclusion criterion that GHS did not support. Specifically, we ensured that the considered software projects:

Table 2
Descriptive statistics of the studied Java and Python projects.

Snapshot	Language	Metric	Sum	Mean	SD	Min	Mdn	Max
First	Java	# Commits	1,070,792	3605.36	10,448.09	100	1262	134,597
		# Contributors	11,822	39.8	40.47	1	27	306
		# Stars	260,554	877.29	1509.73	100	331	11,879
		# Forks	83,520	281.21	474.5	8	127	4,121
	Python	# Commits	460,759	1363.19	2392.21	101	662	21,259
		# Contributors	11,354	33.59	49.3	1	17	402
		# Stars	382,797	1132.54	2639.82	100	339	25,777
		# Forks	66,757	197.51	448.52	4	76	4,461
Second	Java	# Commits	1,052,994	3557.41	10,433.00	100	1259	134,597
		# Contributors	11,681	39.46	40.1	1	27	306
		# Stars	260,526	879.82	1511.66	100	332	11,879
		# Forks	83,338	281.72	475.23	8	126	4,121
	Python	# Commits	460,759	1363.19	2392.21	101	662	21,259
		# Contributors	11,354	33.59	49.3	1	17	402
		# Stars	382,797	1132.54	2639.82	100	339	25,777
		# Forks	66,757	197.51	448.52	4	76	4,461

Table 3
Descriptive statistics for the analyzed dependencies.

Snapshot	Language	% repositories with dependencies	# dependencies[a]					
			Sum	Mean	SD	Min	Mdn	Max
First	Java	100% (297 out of 297)	60,803	204.72	425.36	1	57	3027
	Python	100% (338 out of 338)	14,901	44.09	85.58	1	16	1158
Second	Java	99% (292 out of 296)	63,981	219.11	453.98	1	59	3418
	Python	97% (327 out of 338)	15,762	48.20	89.55	1	18	1133

[a] Excluding repositories without dependencies.

5. **Used Maven, pip, or Poetry as package managers.** This ensured that the included projects used package managers compatible with GitHub's dependency graph (as previously mentioned).

To check whether a software project met Criterion 5, we detected the use of the package manager (*i.e.,* Maven, pip, and Poetry) by looking for the corresponding manifest/lock files in the repository. More specifically, we detected the use of Maven in Java projects by checking — using the *PyGithub* library [25] — for the presence of `pom.xml` files. As far as Python projects are concerned, we detected the use of pip by verifying that they contained the `requirements.txt`, `pipfile`, or `pipfile.lock` files, while we detected the use of Poetry from the presence of `poetry.lock` files. Note that pip and Poetry can leverage other manifest/lock file formats (*e.g.,* `setup.py` and `pyproject.toml`) different from those mentioned above. However, their use is not recommended with GitHub's dependency graph [2]. For this reason, we only focused on `requirements.txt`, `pipfile`, `pipfile.lock`, and `poetry.lock` files for Python projects. We retrieved 1330 Java projects using Maven and 2948 Python projects using pip/Poetry, representing our interest populations. Due to the large size of these populations, we built two statistically significant random samples with a confidence level equal to 95% and a margin of error equal to 5%. As for the population of Java projects using Maven, the required sample size resulted in 298. The sample size required for the Python projects using pip/Poetry was equal to 340. When sampling software projects from the populations of interest, we manually checked (by reading the project's description/README) whether or not each software project was:

6. **Intended for software development.** This avoids selecting books, tutorials, or students' assignments [24].

If a project did not meet Criterion 6 (or the project's description/README was not in English), we discarded it and randomly selected a new candidate project to be included in the sample and reapplied such a criterion. We decided to postpone the check of Criterion 6 at the time of sampling to avoid manually verifying each project in the populations of interest. This design choice does not negatively affect the statistical significance of our samples. This is because if a project were not for software development, it should also be removed from the population of interest.

Finally, we retrieved and then analyzed both dependencies and dependents of each selected project by leveraging GitHub's dependency graph at two different times, so taking two different snapshots: the first snapshot (taken in April 2023—this is the snapshot of our past paper [12]) and second snapshot (September 2024). To do so, and given the lack of GitHub-specific APIs for retrieving information from the dependency graph, we parsed the *dependency graph* pages of each selected project by using a web scraping library, *BeautifulSoup* [26]. Note that, while retrieving and then analyzing dependencies and dependents in the first snapshot, we found three projects (one developed in Java and two developed in Python) that were no longer available after our sampling. Consequently, we had to discard these three projects. Similarly, one Java project was no longer available in second snapshot. We did not replace these projects in the random samples (in either second snapshot or first snapshot) because, after recomputing the required sizes of the random samples in light of the projects discarded for not meeting Criterion 6, our random samples were still large enough. Also, we wanted to keep the set of projects fixed along the two snapshots. That is, our random samples were still statistically significant with a confidence level of 95% and an error margin of 5%. In summary, we ended up with 297 Java and 338 Python projects in first snapshot and 296 Java and 338 Python projects in second snapshot (*i.e.,* the same projects as first snapshot except for one).

In Table 2, we report some descriptive statistics — *i.e.,* sum, mean, Standard Deviation (SD), min, median (Mdn), and max — on the number of commits, contributors, stars, and forks of the projects studied in first snapshot and second snapshot. Regardless of the snapshot, the number of commits, contributors, stars, and forks we summarize in Table 2 were extracted at the time of the query to search software projects on GitHub (April 2023). The only difference between the studied projects in first snapshot and second snapshot concerns one Java project, which was not available anymore at the time of second snapshot. Therefore, while the descriptive statistic values shown in Table 2 are the same in first snapshot and second snapshot for Python

Table 4
Descriptive statistics for the analyzed dependents.

Snapshot	Language	% repositories with dependents	# dependents[a]					
			Sum	Mean	SD	Min	Mdn	Max
First	Java	63% (188 out of 297)	1,761,072	9,367.40	33,147.31	1	342	279,714
	Python	57% (192 out of 338)	307,369	1,600.88	8167	1	48	97,074
Second	Java	67% (197 out of 296)	2,312,381	11,737.97	44,878.61	1	448	360,411
	Python	64% (217 out of 338)	533,284	2,457.53	9526.35	1	67	65,618

[a] Excluding repositories without dependents.

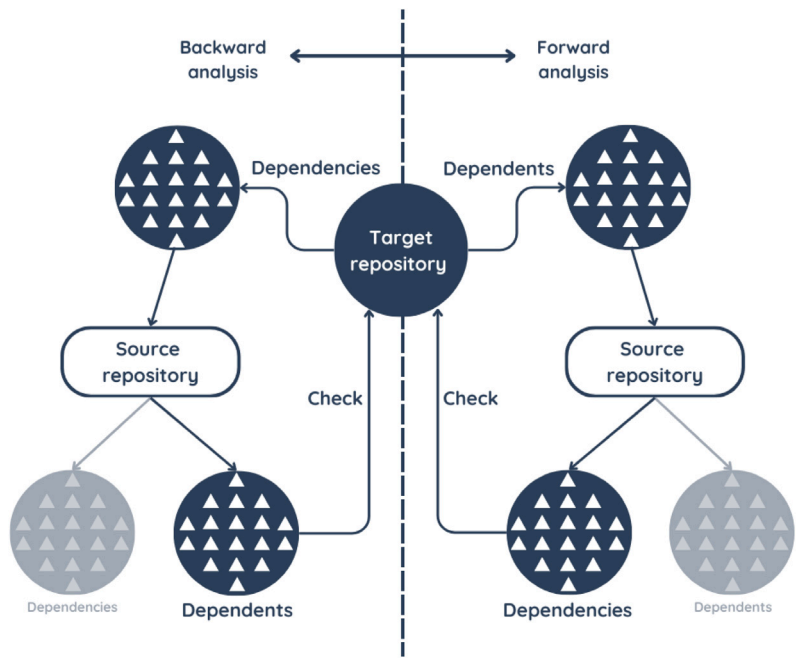

Fig. 1. Backward and forward analysis [12].

projects, there are some slight differences in these values between the two snapshots when considering Java projects.

Tables 3 and 4 summarize the number of dependencies and dependents returned by GitHub's dependency graph, which we analyzed in first snapshot and second snapshot, respectively. The number of dependencies and dependents varies at different times, *i.e.,* April 2023 for first snapshot and September 2024 for second snapshot. For this reason, the same project may have a different number of dependencies or dependents when passing from first snapshot to second snapshot. As shown in Table 3, in first snapshot, we analyzed 60,803 dependencies in Java projects and 14,901 in Python projects. All projects included at least one dependency. As for second snapshot, our analysis covered 63,981 dependencies in Java projects and 15,762 in Python projects. The majority of projects had at least one dependency, with 99% Java projects and 97% Python projects meeting this criterion. As shown in Table 4, in first snapshot, 63% of Java projects and 57% of Python projects had at least one dependent. For second snapshot, our analysis revealed that 66% of Java projects and 64% of Python projects had at least one dependent. Excluding projects without dependents, the average number of dependents per Java and Python project was 11,737.97 and 2457.53, respectively. In total, we analyzed 2,312,381 dependents for Java projects and 533,284 dependents for Python projects.

4. RQ1: To what extent does GitHub's dependency graph provide accurate information?

In this section, we first present the analysis procedure we followed to answer **RQ1** and then the obtained results.

4.1. Analysis procedure

Before detailing the analysis procedure used to answer **RQ1**, we need to introduce the terminology we will use in the remainder of the paper. Specifically, with *target repository*, we refer to the repository that is the object of our analysis. In contrast, with *source repository*, we denote any repository that either depends on or is dependent on a target repository.

To study **RQ1**, we conduct, through Python scripts, three distinct analyses to assess the accuracy of the dependencies and dependents reported in GitHub's dependency graph: *backward analysis*, *forward analysis*, and *manifest/lock file analysis*. We indicate as backward analysis the process aimed at analyzing its dependencies, and as forward analysis the process aimed at analyzing its dependents. The *manifest/lock file analysis* aims at analyzing the package manager files stored in the target repository. These three analyses are designed to evaluate different aspects of the accuracy of GitHub's dependency graph, ensuring a comprehensive assessment thereof. A description of these analyses follows:

- **Backward analysis.** This analysis evaluates the accuracy of the dependencies listed in GitHub's dependency graph. The backward analysis is graphically depicted on the left side of Fig. 1. First, we selected a target repository (*i.e.,* one of the selected projects) and examined its dependencies, as reported in GitHub's dependency graph. Next, for each dependency, we accessed the corresponding source repository and verified whether the target repository was listed among its dependents.

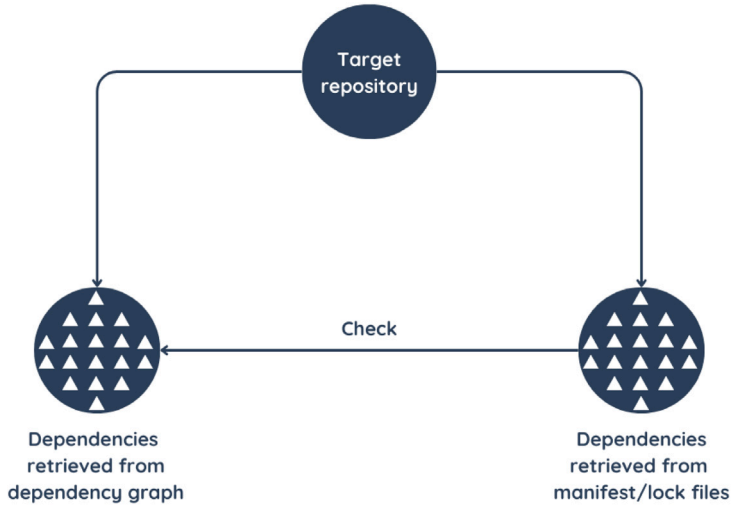

Fig. 2. Manifest/lock file analysis [12].

Table 5
Backward analysis results of Java and Python dependencies.

Snapshot	Detection	Java		Python	
		Sum	%	Sum	%
First	Match	45,707	75.17	12,589	84.48
	Mismatch	15,096	24.83	2,312	15.52
	Total	*60,803*	–	*14,901*	–
Second	Match	46,866	73.25	12,272	77.86
	Mismatch	17,115	26.75	3,490	22.14
	Total	*63,981*	–	*15,762*	–

- **Forward analysis.** It focuses on the accuracy of the dependents reported in GitHub's dependency graph. The right-hand side of Fig. 1 provides a graphical representation of the forward analysis. Starting with a target repository (*i.e.,* one of the projects selected), we first identified its dependents. Then, for each dependent, we accessed its corresponding source repository and checked whether its listed dependencies included the target repository.
- **Manifest/lock file analysis.** This analysis evaluates how accurately GitHub's dependency graph reflects the dependency information contained in manifest/lock files. Fig. 2 shows a graphical representation of the process used to analyze the manifest/lock files. For each target repository, we retrieved its manifest/lock files at the time they were analyzed by GitHub's dependency graph, as we intended to analyze the same file versions. This is possible because, for each dependency, GitHub's dependency graph makes the dependency detection date available. Therefore, we checked out the manifest/lock files on that date and parsed them to extract the list of dependencies they contained. Finally, we checked, through a script, whether the dependencies retrieved from the manifest/lock files matched those in the dependency graph for that target repository. The match took into account (i) repository name, (ii) dependency name, (iii) path to the manifest/lock file, and (iv) dependency version. If a dependency in GitHub's dependency graph was not found in the manifest/lock file (or vice versa), the dependency was labeled as *match not found*, otherwise as *match found*.

4.2. Results

In the following, we report the results for RQ1, according to the backward, forward, and manifest/lock file analyses, respectively.

Table 6
Occurrences of error cases for Java and Python dependencies.

Snapshot	Error case	Java		Python	
		Sum	%	Sum	%
First	Occurrence not found	687	4.55	1612	69.74
	Package not found	1,945	12.89	165	7.14
	Source repository not found	53	0.35	30	1.30
	No link to source repository	12,411	82.21	505	21.84
Second	Occurrence not found	2,557	14.94	2533	72.58
	Package not found	3,522	20.59	541	15.50
	Source repository not found	46	0.27	14	0.40
	No link to source repository	10,990	64.20	402	11.52

4.2.1. Backward analysis

The results of the backward analysis for Java and Python projects are presented in Table 5, for both first snapshot (April 2023) and second snapshot (September 2024). As shown in Table 5, in second snapshot, we examined a total of 79,743 dependencies, with the majority of them originating from Java projects (63,981) and a smaller portion from Python ones (15,762). In terms of accuracy, we can observe that 73.25% of Java dependencies were correctly identified, indicating that there was a match between the dependencies being analyzed and the dependents of those dependencies. On the other hand, the dependency graph produced incorrect information in 26.75% of cases (mismatch). The accuracy for Python projects was greater than for Java, with 77.86% dependencies correctly detected.

As for the error cases for the incorrect detection of dependencies, we identified the following ones:

- ***Occurrence not found.*** It happens when the dependency of a target repository does not list the target repository among its dependents.

Table 7
Repositories, excluding those without dependencies, affected by errors during the backward analysis.

Snapshot	Repositories	Java		Python	
		Sum	%	Sum	%
First	With errors	269	90.57	247	73.08
	Without errors	28	9.43	91	26.92
	Total	*297*	–	*338*	–
Second	With errors	267	91.44	260	79.51
	Without errors	25	8.56	67	20.49
	Total	*292*	–	*327*	–

Table 8
Forward analysis results of dependents.

Snapshot	Detection	Java		Python	
		Sum	%	Sum	%
First	Match	1,701,626	96.62	218,044	70.94
	Mismatch	59,446	3.38	89,325	29.06
	Total	*1,761,072*	–	*307,369*	–
Second	Match	2,266,158	98	477,513	89.54
	Mismatch	46,223	2	55,771	10.46
	Total	*2,312,381*	–	*533,284*	–

- ***Package not found.*** It happens when GitHub's dependency graph divides a target repository into multiple packages, and the dependency of a package does not have that package listed among its dependents.
- ***Source repository not found.*** It happens when the dependency of a target repository is linked to a source repository that does not exist.
- ***No link to source repository.*** It happens when the dependency of a target repository lacks a connection to any source repository.

Table 6 shows the occurrences of the identified error cases across the analyzed projects for both snapshots. In the second snapshot, the most relevant errors for Java repositories were related to *no link to source repository*, which accounted for over 64% of all detected issues. This suggests a systemic problem in how GitHub's dependency graph resolves and associates source repositories for Java packages. Instead, Python projects showed a different error distribution. The most frequent error was *occurrence not found*, representing nearly three-quarters (about 73%) of all Python-related errors. This reflects possible weaknesses in how GitHub tracks backward links from dependencies in the Python ecosystem, perhaps owed to its more dynamic dependency structures.

Finally, we analyzed the number of repositories (excluding those without any dependency), where at least one of the error cases listed above was identified during the backward analysis. The results of this analysis are shown in Table 7 for both snapshots. In the second snapshot, about 91% of Java repositories and 80% of Python repositories incurred at least one incorrect dependency. This means that the dependency graph provided accurate information for all dependencies in only 9% of Java repositories and 20% of Python ones.

Longitudinal analysis. Compared to second snapshot, we examined a lower number of dependencies (75,704) in first snapshot (see Table 5). However, similar to second snapshot, the majority of dependencies originate from Java projects (60,803) and a smaller portion from Python ones (14,901). Moreover, the accuracy of the dependency graph was higher for Python projects (84.48% of dependencies correctly identified) than for Java (75.17% of dependencies correctly identified). To determine whether the proportions of match/mismatch dependencies significantly changed between first snapshot and second snapshot, we used *Fisher's exact test.*[2] This test returned a p-value equal to 8.97×10^{-15} and 2.2×10^{-16} for Java and Python projects, respectively, indicating the presence of statistically significant differences in the proportions of match/mismatch dependencies between the two snapshots for both Java and Python projects. As Table 5 suggests, these statistically significant differences are due to a worsening of the accuracy of the dependency graph, in terms of dependencies, between first snapshot and second snapshot. Finally, as shown in Table 7, in first snapshot, 90.57% of Java and 73.08% of Python repositories had errors in their dependencies. Consequently, the GitHub dependency graph provided accurate dependency information for only 9.43% of Java and 26.92% of Python repositories. To assess whether the proportions of repositories affected or not by errors in their dependencies significantly changed between first snapshot and second snapshot. Fisher's exact test returned a p-value of 0.774 for Java projects and 0.056 for Python ones. Therefore, we cannot conclude that these proportions changed significantly between the two snapshots.

4.2.2. Forward analysis

The results of the forward analysis for Java and Python projects are presented in Table 8, for both first snapshot and second snapshot. As shown in Table 8, in second snapshot, we examined a total of 2,845,665 dependents, with the majority of them originating from Java projects (2,312,381) and a smaller portion from Python ones (533,284). In terms of accuracy, we can observe that 98% of Java dependents were correctly identified, indicating there was a match between the dependents being analyzed and the dependencies of those dependents. Therefore, the dependency graph provided inaccurate information in 2% of Java dependents. Regarding Python projects, the accuracy was lower, with only 89.54% of dependents being correctly detected.

As for the error cases for the incorrect detection of dependents, we identified the cases that follow:

- ***Occurrence not found.*** It happens when the dependent of a target repository does not have that target repository among its dependencies.
- ***Source repository not found.*** It happens when the dependent of a target repository is linked to a source repository that does not exist.
- ***Repository not available.*** It happens when the dependent of a target repository is unavailable due to policy violations (*e.g.,* violations of GitHub's Terms of Service or takedowns under the Digital Millennium Copyright Act).

As done for the backward analysis, we report in Table 9 how often the error cases for the incorrect detection of dependents occurred. In the second snapshot, the majority of errors in Java and Python projects are related to *occurrence not found*, where the expected dependent repository does not appear in GitHub's dependency graph. This suggests a systematic shortcoming in the way GitHub reconstructs the reverse dependencies. Looking at the less frequent error cases, such as *source repository not found* and *repository not available*, it seems that the accuracy of the GitHub dependency graph is also affected by temporal and structural factors, such as repository renaming, deletion, or inaccessibility over time. These issues highlight the inherent fragility of dependency tracking in dynamic and evolving software.

Table 10 reports the number of repositories, excluding those without any dependent, in which at least one of the error cases listed above was detected during the forward analysis. In the second snapshot, about 88% of Java repositories and 99% of Python repositories incurred at least one incorrect dependent. This means that the dependency graph provided accurate information for all dependents in only 12% of Java repositories and 1% of Python repositories.

[2] For any statistical test we ran, we fixed the significance level, α, at 0.05. This is to say that we considered a difference as statistically significant if the p-value returned by the test was less than or equal to α.

Table 9
Occurrences of error cases for Java and Python dependents.

Snapshot	Case	Java		Python	
		Sum	%	Sum	%
First	Occurrence not found	31,885	53.64%	86,292	96.60%
	Source repository not found	27,410	46.11%	3,028	3.39%
	Repository not available	151	0.25%	5	0.01%
Second	Occurrence not found	41,667	90.14%	54,978	98.58%
	Source repository not found	4,016	8.69%	790	1.42%
	Repository not available	540	1.17%	3	<0.01%

Table 10
Repositories, excluding those without dependents, affected by errors during the forward analysis.

Snapshot	Repositories	Java		Python	
		Sum	%	Sum	%
First	With error	163	86.7	150	78.13
	Without errors	25	13.3	42	21.88
	Total	*188*	–	*192*	–
Second	With error	172	87.3	215	99.08
	Without errors	25	12.7	2	0.92
	Total	*197*	–	*217*	–

Table 11
Number of dependencies retrieved from manifest/lock files and dependency graph.

Snapshot	Source	# Dependencies	
		Java	Python
First	Manifest/lock files	68,004	14,858
	Dependency graph	60,803	14,901
Second	Manifest/lock files	75,537	19,569
	Dependency graph	63,981	15,762

Table 12
Comparison between the number of dependencies retrieved from manifest/lock files and dependency graph in the analyzed repositories (excluding those without dependencies and/or for which we failed to retrieve the manifest/lock files).

Snapshot	Case	Java		Python	
		Sum	%	Sum	%
First	Lower	21	7.07	19	5.62
	Equal	84	28.28	284	84.02
	Greater	192	64.65	35	10.36
	Total	*297*	–	*338*	–
Second	Lower	11	3.79	19	5.83
	Equal	59	20.34	255	78.22
	Greater	220	75.86	52	15.95
	Total	*290*	–	*326*	–

Longitudinal analysis. As shown in Table 8, in first snapshot, we examined a total of 2,068,441 dependents (*i.e.,* a lower number of dependents compared to second snapshot). Similar to second snapshot, the majority of dependents originate from Java projects (1,701,626) and a smaller portion from Python projects (307,369). Moreover, the accuracy of the dependency graph, in terms of dependents, was higher for Java projects (96.62% of dependents correctly identified) than Python projects (70.94% of dependents correctly identified). We also ran Fisher's exact test to determine whether the proportions of match/mismatch dependents significantly changed between first snapshot and second snapshot. The test returned a p-value of 2.2×10^{-16}, for Java projects, and 2.2×10^{-16}, for Python ones, indicating statistically significant differences in the proportions of match/mismatch dependents between the two snapshots for both Java and Python projects. By looking at Table 8, it is easy to grasp that the accuracy of the dependency graph, in terms of dependents, improved between the two snapshots. Finally, as shown in Table 10, in first snapshot, mismatch dependents affected 86.7% of Java and 78.13% of Python repositories. Fisher's exact test did not indicate statistically significant differences for Java (p-value of 0.881) while it was the case for Python (p-value of 4.59×10^{-13}).

4.2.3. Manifest/lock file analysis

Table 11 shows the number of dependencies we retrieved during the analysis of manifest/lock files, comparing them with dependencies returned by the dependency graph, for Java and Python projects (in both first snapshot and second snapshot). During the analysis of manifest/lock files, we failed to retrieve six manifest/lock files, four related to Java projects and two related to Python projects, from the corresponding commits (see Section 4.1)—this happened in both first snapshot and second snapshot snapshots for the same manifest/lock files.

As shown in Table 11, we identified 75,537 Java and 19,569 Python dependencies in manifest/lock files in second snapshot. We analyzed the extent to which, across the studied projects, the number of dependencies retrieved from manifest/lock files is smaller, equal to, or greater than the one retrieved from GitHub's dependency graph. The results of this analysis, for both snapshots, are reported in Table 12. In the second snapshot, we found that, for the vast majority of Java repositories (about 76%), the manifest/lock files revealed more dependencies than those captured by GitHub's dependency graph. This suggests that GitHub's extraction mechanisms may be systematically under-reporting dependencies in Java, likely due to challenges in parsing Maven's hierarchical and variable-driven configuration structures. Conversely, the picture is different for Python where, for the vast majority of repositories (about 78%), the number of dependencies matched exactly between the two sources, indicating a stronger alignment between the manifest/lock files and GitHub's dependency graph. Discrepancies were relatively limited. These findings reinforce the view that GitHub dependency extraction performs more reliably in Python projects, where dependency declarations are typically simpler and more centralized, while Java's more complex configuration introduces higher chances of omission or misinterpretation. Moreover, for both Java and Python projects, in light of our previous analyses, we can better interpret when fewer dependencies are found in the manifest/lock files than in GitHub's dependency graph. Specifically, this is likely due to situations where dependencies were removed from the project's manifest/lock files but are still present in GitHub's dependency graph due to outdated indexing or delayed updates. This reinforces the idea that GitHub's dependency graph does not always promptly reflect changes in a project's configuration and may retain obsolete dependencies that no longer exist in the actual codebase. Such stale data can distort the representation of a project's current dependency structure and must be considered when using the dependency graph for research or analysis purposes.

Finally, to understand to what extent the information from manifest/lock files is consistent with that retrieved from the dependency graph, in Table 13, we report the number and percentage of dependencies retrieved from manifest/lock files matching those retrieved from the dependency graph. In second snapshot, the dependencies of Java repositories retrieved from manifest/lock files matched those from GitHub's dependency graph in about 76% of cases. This percentage drops to 67% for the dependencies of Python repositories.

Longitudinal analysis. As Table 11 shows, we identified 68,004 Java and 14,858 Python dependencies in manifest/lock files in first snapshot. In comparison with the dependencies extracted from the dependency graph, Java repositories showed 7201 additional dependencies declared in manifest/lock files, while Python repositories had 43 fewer dependencies declared in manifest/lock files. Moreover, in first snapshot, the number of dependencies retrieved from manifest/lock files was greater, equal, and lower than those from GitHub's dependency graph in 64.65%, 28.28%, and 7.07% of Java repositories, respectively

Table 13
Match between the dependencies retrieved from the manifest/lock files and those from the dependency graph.

Snapshot	Match	Java		Python	
		Sum	%	Sum	%
First	Found	52,490	77.19	14,220	95.71
	Not found	15,514	22.81	638	4.29
	Total	*68,004*	–	*14,858*	–
Second	Found	57,691	76.37	13,114	67.01
	Not found	17,846	23.63	6,455	32.99
	Total	*75,537*	–	*19,569*	–

(see Table 12). As for Python repositories, the number of dependencies retrieved from manifest/lock files was greater, equal, and lower than those from GitHub's dependency graph in 10.36%, 84.02%, and 5.62% of cases, respectively. Finally, in second snapshot, the dependencies of Java repositories retrieved from manifest/lock files matched those from GitHub's dependency graph in 77.19% of cases (see Table 13). This percentage increases to 95.71% for the dependencies of Python repositories. To determine whether the proportions of the matched/unmatched manifest/lock files changed significantly between first snapshot and second snapshot, we leveraged Fisher's exact test. For Java, the test returned a p-value equal to 3×10^{-04}, indicating a statistically significant difference in the proportions of matched/unmatched manifest/lock files between the two snapshots. For Python, the test returned a p-value 2.21×10^{-16}, suggesting again a statistically significant difference in the proportions. As Table 13 suggests, these differences are due to a substantial drop in the match between the dependencies declared in manifest/lock files and those retrieved from the dependency graph when passing from first snapshot to second snapshot for both Java and Python projects.

Answer to RQ1: GitHub's dependency graph reports inaccurate information, in either the dependencies or dependents, for the majority of the projects analyzed. This derives from up to ~25% in first snapshot and ~27% in second snapshot of incorrect dependencies, and ~29% in first snapshot and ~10% in second snapshot of incorrect dependents. The incorrect dependencies occur with different error types, related to dependency occurrences, packages, source repositories not found, or wrong links are evident from a mismatch between the GitHub dependency graph and the manifest/lock files. Moreover, the dependency information from manifest/lock files and GitHub's dependency graph does not always match: 77.19% in first snapshot and 76.37% in second snapshot of Java dependencies have a match, while Python dependencies have a match in 95.71% in first snapshot and 67.01% in second snapshot of cases. For Java, the percentage of errors does not consistently decrease over the two observed snapshots, but it does for Python. Anyway, imprecise analyses persist in the dependency graph computation.

5. RQ2: What are the likely causes for the error cases occurring in GitHub's dependency graph?

In this section, we present the analysis procedure we defined to answer **RQ2**, followed by the obtained results.

5.1. Analysis procedure

To answer **RQ2**, we analyzed the occurrences of the error cases we identified in our previous analyses — *i.e.,* backward, forward, and manifest/lock file analyses — to determine the likely root causes behind them. Given that the total set of errors is too large to be manually analyzed exhaustively, for each of the above-mentioned analyses, we built a statistically significant sample of the error case occurrences, with a confidence level of 95% and a margin of error of 5%. To ensure a balanced representation of the different error cases across Java and Python projects, we performed a stratified sampling consisting of two strata: (i) the programming language and (ii) the error case.

Below, we describe the analysis procedure for the error cases that occurred in the backward, forward, and manifest/lock file analyses, respectively:

- **Backward analysis.** When sampling the error cases that occurred in the backward analysis, we excluded the *source repository not found* and *no link to source repository* error cases because, given the intrinsic nature of these error cases (missing link to repository to be analyzed), we could not investigate deeply the root causes behind them. Therefore, starting from a population of 9153 errors (*i.e.,* 5090 errors related to *occurrence not found* plus 4063 errors related to *package not found* across Java and Python projects, see Table 6), we built a significant sample resulting in 369 error case occurrences, including 205 errors related to *occurrence not found* and 164 errors related to *package not found*, as shown in Table 14. To analyze the *occurrence not found* error case, three couples of authors (hereafter referred to as annotators) manually assessed the correctness of the reported dependencies, following a structured process:

  1. Access the list of dependents for each source repository using the provided GitHub link.
  2. Determine the correct package:
     - If multiple packages are available, select (if any) the correct package corresponding to the dependency under analysis from the drop-down menu (see Fig. 3).
     - If only one package is available, check whether the package name reported in "Repositories that depend on <package name>", corresponds to the dependency under analysis (see the top left corner of Fig. 4).
  3. Determine the likely root cause(s), and whether the target repository is not listed as a dependent.
     - If the target repository name appears to be changed or different from the expected one, take note of the discrepancy and then search for the correct target repository name within the list of dependents reported by GitHub.
     - If the target repository is not found, record a negative match.
  4. Access the source code of the target repository and inspect its manifest/lock files to understand whether the dependency is explicitly declared in the repository's manifest/lock files.
  5. If the dependency is found in the manifest/lock files but is missing from GitHub's dependency graph, take note of this.

  Throughout the process, the annotators recorded their observations on a dedicated evaluation sheet. After completing the annotation process, a third researcher manually analyzed the notes recorded by each annotator to extract meaningful error labels. If any disagreement between the two annotators was found, the third researcher would deeply investigate the occurrence and extract the final label. We measured the agreement between the pairs of annotators by using *Krippendorff's alpha*. The resulting

Table 14
Sampling of error case occurrences.

Analysis	Error case	Java sample	Python sample	Total sample
Backward	Occurrence not found	103	102	205
	Package not found	142	22	164
	Total	*245*	*124*	*369*
Forward	Occurrence not found	156	206	362
	Source repository not found	15	10	25
	Repository not available	10	3	13
	Total	*181*	*219*	*400*
Manifest/lock file	Not found	278	101	379

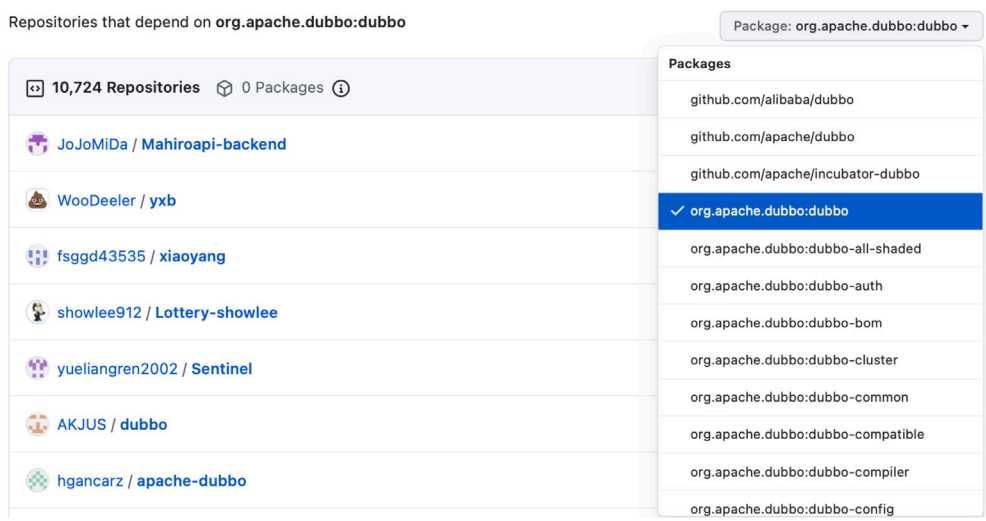

Fig. 3. Example of package drop-down menu of GitHub dependency graph.

value was $\alpha = 0.862$, which indicates a high level of reliability and a strong agreement between the annotators. As for the *package not found* error case, we examined whether these occurrences were attributable to GitHub's processing limits [27]. Given that GitHub imposes constraints on dependency resolution, some errors might stem from these limitations rather than inaccuracies in the dependency graph itself. To assess this, we automatically analyzed, using a script included in the replication package [28], the identified occurrences to determine if they aligned with known processing restrictions. If an error could be linked to these constraints, we categorized it accordingly.

- **Forward analysis.** Given the 101,994 errors that occurred in the forward analysis (46,223 coming from Java projects and 55,771 from Python ones, as shown in Table 9), the resulting significant sample consisted of 400 errors, including 362, 25, 13 errors belonging to *occurrence not found*, *source repository not found*, and *not available*, respectively (see Table 14). To analyze the errors identified during the forward analysis, we performed an automated inspection of the version history of manifest/lock files using a script included in the replication package [28]. For each dependent of a target repository, we accessed the source repository and then checked whether the dependent under analysis was ever present and whether it was later removed from the source repository's dependencies.
- **Manifest/lock file analysis.** Starting from the 24,301 errors that occurred in the manifest/lock file analysis (17,846 resulting from Java projects and 6455 resulting from Python ones, see Table 13), the built significant sample consisted of 379 occurrences, as detailed in Table 14. The goal of this analysis is to assess the likely root causes of the errors encountered when there was a mismatch between the dependencies recorded in the manifest/lock files and those reported in GitHub's dependency graph. To conduct the analysis, three pairs of authors (*i.e.,* annotators) examined whether the dependencies recorded in the manifest/lock files contained any inconsistencies, such as missing information or the use of variables (*e.g.,* `${james.groupId}:james-server-data-memory`). If an inconsistency was found, the annotators recorded it. Next, the annotators accessed the commit history of the repository and selected the option to view the code at the specific commit for which GitHub's dependency graph was analyzed. This step allowed us to retrieve the exact version of the manifest/lock file that was used in GitHub's dependency detection process. We then manually examined the manifest/lock file's content to check for the dependency name's presence and identify potential causes for mismatches between the dependency graph and the actual file content. For the manifest/lock file analysis, the annotators recorded their evaluations in a dedicated sheet. At the end of the annotation process, a third researcher reviewed the annotations and accompanying notes to extract consistent and meaningful labels for each case. In the presence of disagreements, the third researcher conducted a detailed inspection to determine the final classification. We assessed the reliability of the manual labeling process using Krippendorff's alpha. The resulting value was $\alpha = 0.973$, indicating an excellent level of agreement between the annotators and a highly reliable annotation process.

5.2. Results

In the following, we report the results for RQ2 for the backward, forward, and manifest/lock file analyses, respectively.

5.2.1. Backward analysis

We identified the following likely root causes for errors related to *occurrence not found*:

Fig. 4. Example of a single package of GitHub dependency graph.

- ***Dependent not yet indexed.*** The dependent repository is not listed in GitHub's dependency graph at the time of analysis, likely due to indexing delays or incomplete updates.
- ***Name of the repository changed.*** The dependent repository name has been modified since GitHub last indexed it, leading to a mismatch between the recorded dependent and the actual project.
- ***Owner of the repository changed.*** The dependent repository had been transferred to a different owner, which affects how dependencies are tracked and displayed in GitHub's dependency graph.
- ***Package name differs from the actual one.*** The package name of the dependent repository reported in GitHub's dependency graph did not match the actual name used in the dependent repository.
- ***Manifest/lock file not found.*** The manifest/lock file where the dependent repository declares the dependency (identified by the dependency graph) has not been found.
- ***Archived repository.*** The dependent repository has been archived and is now read-only. Consequently, the dependency graph is not enabled anymore for that repository.
- ***Inconsistency (hyphen "–" vs. underscore "_").*** The package name of the dependent's repository reported in GitHub's dependency graph did not match the actual name of the repository due to misuse of hyphens/underscores. For example, the package name reported in GitHub's dependency graph is "parse-type", but the actual name of the package is "parse_type".
- ***Dependent not in manifest/lock files.*** The dependent is not found in the manifest/lock files.
- ***Unknown.*** When the likely reason for an error could not be clearly identified based on the available data.

Table 15 shows how often each of the above-mentioned likely root causes was identified. For Java repositories, the most likely error cause was *manifest/lock file not found*, accounting for 22.32% of the cases. Notably, the most frequent cause among Java repositories was the absence of a manifest or lock file, suggesting that GitHub's dependency graph may have limitations in correctly locating or indexing these files for Java projects. Python projects were more often affected by inconsistencies in package naming. The *package name differs from the actual one* issue alone accounted for more than a third (35.66%) of the observed Python errors, highlighting the sensitivity of the graph builder to naming conventions and metadata accuracy. We also observed that repository ownership or name changes contributed non-negligibly to errors, especially in Python. These suggest that GitHub's indexing and tracking mechanisms can be brittle in the face of project evolution, such as renaming or transferring ownership, which can affect the reliability of dependency resolution over time. Overall, the analysis indicates that error causes are ecosystem-specific and that the underlying mechanisms of the dependency graph may be more prone to failure under certain structural or metadata-related conditions, particularly in dynamically evolving projects.

As far as the *package not found* error case, we identified two likely causes:

- ***Exceeds max number of manifest/lock files.*** It indicates issues with packages not being found due to exceeding the maximum number of manifest/lock files allowed per GitHub repository.

Table 15
Likely causes for the *occurrence not found* error case encountered in the backward analysis.

Likely cause	Java		Python	
	Sum	%	Sum	%
Dependent not yet indexed	15	13.39	18	13.95
Name of the repository changed	20	17.86	6	4.65
Owner of the repository changed	18	16.07	34	26.36
Package name differs from the actual one	24	21.43	46	35.66
Manifest/lock file not found	25	22.32	1	0.78
Archived repository	1	0.89	2	1.55
Inconsistency (hyphen "–" vs. underscore "_")	0	0	14	10.85
Dependent not in manifest/lock files	3	2.68	7	5.43
Unknown	6	5.36	1	0.78
Total	*112*	–	*129*	–

Table 16
Likely causes for the *package not found* error case encountered in the backward analysis.

Likely cause	Java		Python	
	Sum	%	Sum	%
Exceeds max number of manifest/lock files (150)	106	74.65	0	0.00
No processing limit violated	36	25.35	22	100.00
Total	*142*	–	*22*	–

- ***No processing limit violated.*** It indicates that no processing limit was violated by the repository.

Table 16 shows how often each of the above-mentioned likely root causes was identified. The most frequent issue among Java repositories was the violation of GitHub's limit on the number of manifest/lock files, which occurred in 74.65% of the cases. In contrast, all Python repositories (100%) and a minority of Java repositories (25.35%) exhibited no identifiable processing limit violation. It is worth noting that, beyond the *exceeds max number of manifest/lock files* constraint, we also examined other documented processing limits [27] — such as file size or file count thresholds — but found no instances of those limits being exceeded. Once again, this suggests that the observed issues are closely tied to package manager characteristics. In particular, Maven-based Java projects commonly define dependencies across multiple manifest files organized hierarchically, which increases the likelihood of exceeding GitHub's processing thresholds. While such hierarchical structures are technically possible in Python, they are less common in practice.

5.2.2. Forward analysis

We identified the following likely root causes for errors related to the forward analysis:

- ***Dependency in the DG, but not in the manifest/lock files.*** This case occurs when a dependency is listed in GitHub's dependency graph (DG), but no corresponding declaration is found in the repository's manifest/lock files.

Table 17
Likely causes for the errors that occurred in the forward analysis.

Likely cause	Java		Python	
	Sum	%	Sum	%
Dependency in the DG, but not in the manifest/lock file	6	3.59	54	26.6
Dependency deleted, dependent of target repository not upgraded	120	71.85	92	45.32
Dependency in the manifest/lock files, but not in DG	29	17.37	6	2.96
DG of source repository is not enabled	2	1.2	48	23.65
The file disappears from the commit history, dependent of target repository not upgraded	3	1.8	1	0.49
Error resolved	7	4.19	2	0.99
Total	*167*	–	*203*	–

- ***Dependency deleted, dependent of target repository not upgraded.*** Here, the dependency was previously declared in manifest/lock files but was later removed. However, GitHub's dependency graph still shows the corresponding dependent of the target repository.
- ***Dependency in manifest/lock files, but not in DG.*** This refers to a dependency that is correctly listed in manifest/lock files but is missing in GitHub's dependency graph. This suggests a failure in GitHub's dependency extraction or update mechanisms.
- ***DG of source repository is not enabled.*** The dependency is listed in the manifest/lock files of the source repository, but GitHub's dependency graph is not enabled for that source repository.
- ***The file disappears from the commit history, dependent on the target repository not being upgraded.*** This case refers to situations where the manifest/lock files containing the dependency were deleted in the repository's commit history, but GitHub's dependency graph still shows the corresponding dependent of the target repository.
- ***Error resolved.*** We observed a match between the dependent and the dependency in GitHub's dependency graph. This is because GitHub solved the error meanwhile.

The results of the qualitative analysis for the errors detected during forward analysis are presented in Table 17. In Java repositories, the most frequent error cause encountered in the forward analysis was *dependency deleted, dependent of target repository not upgraded*, which accounted for over 70% of the cases. This suggests that GitHub's dependency graph may retain outdated dependency relationships even after the relevant declarations have been removed from the manifest/lock files. Another frequent issue in Java was the presence of dependencies in manifest/lock files that were not reflected in the dependency graph, indicating possible failures in GitHub's extraction or update mechanisms. Python repositories showed a similar trend, with *dependency deleted, dependent of target repository not upgraded* as the most common cause (about 45%). However, other causes were more prominent than in Java. Notably, over a quarter of the Python errors involved dependencies appearing in the dependency graph without a corresponding declaration in the manifest/lock files.

Overall, the results suggest that many errors in the forward analysis are due to failures in automatically updating the dependency graph, even though manifest/lock files were modified correctly in commits. This raises questions about the reliability of automatic dependency propagation in GitHub's dependency graph.

5.2.3. Manifest/lock file analysis

We identified the following likely root causes for the error that occurred in the manifest/lock files analysis:

- ***Missing group ID.*** The dependency declaration in manifest/lock files lacked the necessary group identifier (only for Java projects using Maven).
- ***Group ID defined by a variable.*** Instead of a fixed value, the dependency declaration in manifest/lock files uses a variable that GitHub's dependency graph seems not to resolve correctly (only for Java projects using Maven).

Table 18
Likely causes for the errors that occurred in the manifest/lock file analysis.

Likely cause	Java		Python	
	Sum	%	Sum	%
Missing group ID	45	13.64	0	0
Group ID defined by variable	97	29.39	0	0
Version defined by variable	100	30.3	0	0
Artifact ID defined by variable	9	2.73	0	0
Version numbering schema issue	10	3.03	90	84.91
Manifest/lock files not found in commit history	49	14.85	15	14.15
Usage of scopes	9	2.73	0	0
Not found in the manifest/lock file	9	2.73	0	0
Unknown	2	0.61	1	0.94
Total	*330*	–	*106*	–

- ***Version defined by variable.*** The manifest/lock files report a dependency declaration where the version is not a fixed value but is defined by a variable that GitHub's dependency graph seems not to resolve correctly (only for Java projects using Maven).
- ***Artifact ID defined by variable.*** The manifest/lock file reports a dependency declaration where the artifact ID is not a fixed value but is defined by a variable that GitHub's dependency graph seems not to resolve correctly (only for Java projects using Maven).
- ***Version numbering schema issues.*** Issues in how the versions are specified (*e.g.,* ranges, wildcards, or custom numbering conventions), which GitHub's dependency graph seems not to understand.
- ***Manifest/lock files not found in the commit history.*** The commit in which GitHub's dependency graph inferred the dependency from manifest/lock files disappeared.
- ***Usage of scopes.*** The dependency is declared in a scope that limits its visibility/applicability (only for Java projects using Maven). For example, the dependency is defined only for development (`dev` scope) or testing purposes (`test` scope). Such a dependency seems not to be recognized correctly by GitHub's dependency graph.
- ***Not found in the manifest/lock file.*** The dependency is not found in the manifest/lock file of the repository.
- ***Unknown.*** Cases where the likely reason for the error could not be clearly identified based on the available data.

The results of the qualitative analysis for the errors detected during manifest/lock files analysis are presented in Table 18. In Java repositories, the most frequent issues involved dynamic references that GitHub's dependency graph appeared unable to resolve: *version defined by variable* and *group ID defined by variable*. Specifically, dependencies defined using variables for the version or group ID were the dominant causes, accounting for nearly 60% of the observed errors. These highlight a key limitation in the dependency graph parsing logic when dealing with Maven-based Java projects, where such variable declarations are common practice. In contrast, the vast majority of errors in Python repositories (about 85%) stemmed from version specification formats that GitHub's dependency graph failed to interpret correctly.

Based on the results, we can postulate that many errors in the manifest/lock files analysis are due to failures of the dependency graph in automatically extracting the information from manifest/lock files. These failures originate from GitHub's dependency graph limitations [27] in parsing the information from manifest/lock files (*e.g.,* version numbering schema) or resolving dynamic references (*e.g.,* versions, group ID, and artifact ID defined through variables).

Answer to RQ2: The observed results suggest that most errors in GitHub's dependency graph are caused by limitations in dealing with the information from GitHub metadata and manifest/lock files. That is, GitHub's dependency graph might not update its dependency information after a change involving a given repository. This change might involve GitHub metadata (*e.g.,* the name of the repository or its owner) or the manifest/lock files declaring project dependencies. The limitations of GitHub's dependency graph also involve the extraction of information from manifest/lock files (*e.g.,* parsing and resolution of dynamic references).

6. Discussion

Our study provides evidence that GitHub's dependency graph presents accuracy issues, which affect both backward (dependency) and forward (dependent) analyses. While some improvements are observed in the accuracy of dependents between the two snapshots — possibly due to improved indexing or metadata refinement — the overall accuracy of dependencies has worsened over time. The consistency between GitHub's dependency graph and manifest/lock files also declined, particularly for Python projects, where the match rate dropped sharply. These results could be due to a lack of robustness in GitHub's dependency extraction mechanisms and also highlight dependency graph weaknesses to repository-level changes.

In the following, we present the potential implications of the results obtained in our study and discuss possible threats that could affect their validity.

6.1. Implications

For what concerns **researchers**, our study points out the imprecision of GitHub's dependency graph. Such imprecision can impact the results of research leveraging the dependency graph. For example, if a study aims to analyze all projects using a given framework, library, or tool, the investigated population may be at the same time incomplete and also contain wrong data points. For this reason, research work leveraging it for different purposes, for example studying software ecosystems or SBOMs [9,29] should, at the minimum, perform a manual validation of at least a significant sample of the extracted graph (if not the entire one) to determine the extent to which the work's findings are subject to imprecision. This, for example, was done in the work by Nocera et al. [9,29]. In addition to that, researchers willing to achieve a better level of accuracy should leverage further sources for dependencies—*e.g.,* by implementing more accurate analyzers for manifest/lock files.

Finally, based on the taxonomy of dependency graph errors, researchers could devise approaches to circumvent the identified imprecisions by complementing the dependency graph with further, more accurate analyses, overcoming the found limitations.

For what concerns **practitioners**, the inaccuracy of GitHub's dependency graph can impact the recommender systems leveraging it. This is, in particular, the case of tools such as Dependabot or the GitHub SBOM generator. For what concerns Dependabot, a possible effect of an inaccurate dependency graph is that developers may not be able to update dependencies when needed, leading to potential vulnerability exposures. For what concerns the SBOM generator, an inaccurate dependency graph may lead to SBOMs with missing or wrong dependencies. This could have consequences if the SBOM is used, in turn, for vulnerability assessment, or because this could lead to licensing violations, and therefore legal consequences.

Our findings are also relevant for the **developers** and **maintainers of the GitHub** infrastructure. Notably, GitHub has recently distributed a survey to its users, seeking feedback on the usage of its dependency graph feature to guide future improvements [30]. In this context, our study provides a systematic categorization of the error types affecting both dependencies and dependents within GitHub's dependency graph. These documented issues can offer valuable insights into recurring failure modes and may directly inform efforts to enhance the accuracy and reliability of this critical feature.

6.2. Threats to validity

In this section, we discuss the threats that might affect our results. Based on the validity schema by Wohlin et al. [31], they fall into the following categories: construct, conclusion, and external. We do not discuss threats to internal validity since we do not investigate causality.

Construct Validity. Threats to construct validity concern the relation between theory and observation [31]. In our study, such a kind of threat regards the quantitative analyses — *i.e.,* backward, forward, manifest/lock file analyses — we performed to assess the accuracy of GitHub's dependency graph. We relied on widely adopted libraries to analyze the relevant artifacts, thoroughly tested our scripts, and complemented the quantitative analysis with a qualitative one to categorize the error's root causes. It is important to point out that we assume as "ground truth" the dependencies contained in manifest/lock files, as they are the ones created by the projects' developers. This is because our goal is to determine whether mismatches are introduced when the dependency graph is created from them. This does not mean that such dependencies are correct. However, determining whether program dependencies need to be fixed or upgraded is out of the scope of this work. A further threat to construct validity concerns mistakes and subjectivity in the qualitative analysis. To mitigate this threat, each instance has been classified by two independent annotators, and conflict resolution has been performed on cases of inconsistent labeling. Furthermore, the labeling reliability has been assessed using Krippendorff's alpha inter-rater agreement.

Conclusion Validity. Threats to conclusion validity concern the statistical conclusions, including sample composition and size. The selection of a subset of projects to be analyzed might lead to spurious results. To mitigate this threat, we employed sampling strategies that are systematic and reproducible. Specifically, we studied statistically significant samples of Java and Python projects hosted on GitHub and selected by following the criteria described in Section 3.2. For a similar reason, when studying RQ2, we built a statistically significant stratified sample (by language and error types) of the observed error case occurrences.

External Validity. Threats to external validity concern the generalizability of results [31]. One key limitation of our study is that the results may not be generalized to software projects (hosted in GitHub) written in languages other than Java and Python. Different programming languages often exhibit distinct ecosystem characteristics, dependency management conventions, and levels of toolchain support—all of which can influence the accuracy of GitHub's dependency graph. Notably, our findings reveal significant differences in accuracy even between Java and Python, underscoring the importance of further research to understand how these variations manifest across a wider range of programming languages, among those for which GitHub creates dependency graphs, *e.g.,* C#, JavaScript, or Ruby. Another threat to external validity concerns the application of inclusion and exclusion criteria (see Section 3.2). While inclusion and exclusion

criteria are necessary to ensure the relevance and quality of the selected repositories, they inherently restrict the target population to which the results can be generalized. For example, focusing solely on popular or highly active repositories may lead to conclusions that do not hold for less popular or inactive repositories. Moreover, it is possible that we excluded some stable yet inactive projects, potentially influencing our findings. Finally, our results may not fully capture the current accuracy of GitHub's dependency graph, as some inaccuracies might have been corrected over time. However, our analysis in two snapshots suggests that the accuracy of dependencies between the two snapshots decreased, and the qualitative analysis performed indicated that there are elements of manifest/lock files that the GitHub dependency graph builder is unable to capture.

7. Conclusion

We presented the results of a mining study to assess the accuracy of GitHub's dependency graph in Java and Python open-source software projects. To this end, we performed a: (i) *backward analysis*, focusing on the accuracy of a given repository's dependencies, as reported in the dependency graph; (ii) *forward analysis*, focusing on the accuracy of the dependents, as reported in the dependency graph of a given repository; and (iii) *manifest/lock file analysis* focusing on the correspondence between the information reported in the dependency graph and what was reported in the corresponding manifest/lock files. The analysis has been repeated twice, considering a snapshot of the dependency graphs extracted in April 2023 and a snapshot extracted in September 2024.

The analyses reported at least one inaccuracy in the majority of the analyzed projects, and the accuracy of the dependency graphs has not improved over the two snapshots. The quantitative analysis has been complemented with a qualitative one, in which we categorized the root causes of the observed dependency graph errors. Such root causes point out limitations in how the dependency graph is created, particularly how manifest/lock files are analyzed. The results of our study have some practical implications. For example, imprecision in GitHub's dependency graph could affect recommenders (*e.g.,* Dependabot) and other tools based on it, such as GitHub's SBOM generation tool.

Future work around this research includes (i) extending the study to further programming languages, and (ii) understanding the extent to which imprecision in GitHub's dependency graph affects past studies based on it.

CRediT authorship contribution statement

Daniele Bifolco: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Simone Romano:** Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Conceptualization. **Sabato Nocera:** Writing – review & editing, Writing – original draft, Visualization, Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Rita Francese:** Conceptualization. **Giuseppe Scanniello:** Conceptualization. **Massimiliano Di Penta:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This project has been financially supported by the European Union NEXTGenerationEU project and by the Italian Ministry of the University and Research MUR, a Research Projects of Significant National Interest (PRIN) 2022 PNRR, project n. D53D23017310001 entitled "Mining Software Repositories for enhanced Software Bills of Materials (MSR4SBOM)". Daniele Bifolco is partially funded by the PNRR DM 118/2023 Italian Grant for Ph.D. scholarships.

Data availability

I have shared the link to the data and scripts used to perform the study, in the paper (also referred as Replication Package). https://doi.org/10.5281/zenodo.15412179.

References

[1] Stack Overflow, 2022 developer survey, 2022, https://survey.stackOverflow.co/2022.
[2] GitHub, About the dependency graph, 2025, https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/about-the-dependency-graph.
[3] GitHub, Dependency graph supported package ecosystems, 2025, https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/dependency-graph-supported-package-ecosystems.
[4] GitHub, Dependabot, 2023, https://docs.github.com/en/code-security/dependabot.
[5] Eric Tooley, Erin Havens, A Smarter Quieter Dependabot, 2023, https://github.blog/2023-01-12-a-smarter-quieter-dependabot/.
[6] Mahmoud Alfadel, Diego Elias Costa, Emad Shihab, Mouafak Mkhallalati, On the use of dependabot security pull requests, in: Proceedings of International Conference on Mining Software Repositories, IEEE, 2021, pp. 254–265.
[7] Runzhi He, Hao He, Yuxia Zhang, Minghui Zhou, Automating dependency updates in practice: An exploratory study on GitHub dependabot, IEEE Trans. Softw. Eng. 49 (8) (2023) 4004–4022.
[8] João Eduardo Montandon, Marco Tulio Valente, Luciana L. Silva, Mining the technical roles of GitHub users, Inf. Softw. Technol. 131 (2021) 106485.
[9] Sabato Nocera, Simone Romano, Massimiliano Di Penta, Rita Francese, Giuseppe Scanniello, Software bill of materials adoption: A mining study from GitHub, in: 2023 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2023, pp. 39–49, http://dx.doi.org/10.1109/ICSME58846.2023.00016.
[10] Federica Pepe, Fiorella Zampetti, Antonio Mastropaolo, Gabriele Bavota, Massimiliano Di Penta, A taxonomy of self-admitted technical debt in deep learning systems, in: 2024 IEEE International Conference on Software Maintenance and Evolution, ICSME, 2024, pp. 388–399, http://dx.doi.org/10.1109/ICSME58944.2024.00043.
[11] Federica Pepe, Vittoria Nardone, Antonio Mastropaolo, Gabriele Bavota, Gerardo Canfora, Massimiliano Di Penta, How do hugging face models document datasets, bias, and licenses? An empirical study, in: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, ICPC '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 370–381, http://dx.doi.org/10.1145/3643916.3644412.
[12] Daniele Bifolco, Sabato Nocera, Simone Romano, Massimiliano Di Penta, Rita Francese, Giuseppe Scanniello, On the accuracy of GitHub's dependency graph, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, ACM, New York, NY, USA, 2024, http://dx.doi.org/10.1145/3661167.3661175.
[13] Neal Robert Haddaway, Alexandra Mary Collins, Deborah Coughlin, Stuart Kirk, The role of google scholar in evidence reviews and its applicability to grey literature searching, PLoS One 10 (2015) 1–17, http://dx.doi.org/10.1371/journal.pone.0138237.
[14] Hamid Mohayeji, Andrei Agaronian, Eleni Constantinou, Nicola Zannone, Alexander Serebrenik, Investigating the resolution of vulnerable dependencies with dependabot security updates, in: Proceedings of 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, IEEE, 2023, pp. 234–246.
[15] Tajkia Rahman Toma, Cor-Paul Bezemer, An exploratory study of dataset and model management in open source machine learning applications, in: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI, 2024, pp. 64–74.
[16] Zhimin Zhao, Yihao Chen, Abdul Ali Bangash, Bram Adams, Ahmed E Hassan, An empirical study of challenges in machine learning asset management, Empir. Softw. Eng. 29 (4) (2024) 98.
[17] Akalanka Galappaththi, Sarah Nadi, Christoph Treude, An empirical study of API misuses of data-centric libraries, in: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2024, pp. 245–256.
[18] Kai Gao, Weiwei Xu, Wenhao Yang, Minghui Zhou, PyRadar: Towards automatically retrieving and validating source code repository information for PyPI packages, Proc. the ACM Softw. Eng. 1 (FSE) (2024) 2608–2631.
[19] Samuel Idowu, Yorick Sens, Thorsten Berger, Jacob Krüger, Michael Vierhauser, A large-scale study of ml-related python projects, in: Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, 2024, pp. 1272–1281.
[20] Ekaterina Ivanova, Natalia Stakhanova, Bahman Sistany, Adversarial analysis of software composition analysis tools, in: International Conference on Information Security, Springer, 2024, pp. 161–182.

[21] Sheng Yu, Wei Song, Xunchao Hu, Heng Yin, On the correctness of metadata-based SBOM generation: A differential analysis approach, in: 2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, IEEE, 2024, pp. 29–36.
[22] PYPL, 2025 PYPL index, 2025, https://pypl.github.io/PYPL.html.
[23] Ozren Dabic, Emad Aghajani, Gabriele Bavota, Sampling projects in GitHub for MSR studies, in: Proceedings of International Conference on Mining Software Repositories, IEEE, 2021, pp. 560–564.
[24] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, Daniela Damian, The promises and perils of mining GitHub, in: Proceedings of Mining Software Repositories, ACM, 2014, pp. 92–101.
[25] Vincent Jacques, Pygithub, 2023, https://pygithub.readthedocs.io/en/stable/.
[26] Leonard Richardson, BeautifulSoup, 2023, https://www.crummy.com/software/BeautifulSoup/.
[27] GitHub, Troubleshooting the dependency graph, 2025, https://docs.github.com/en/enterprise-server@3.9/code-security/supply-chain-security/understanding-your-software-supply-chain/troubleshooting-the-dependency-graph#are-there-limits-which-affect-the-dependency-graph-data.
[28] Daniele Bifolco, Sabato Nocera, Simone Romano, Rita Francese, Giuseppe Scanniello, Massimiliano Di Penta, An empirical study on the accuracy of GitHub's dependency graph and the nature of its inaccuracy: A replication package, 2025, http://dx.doi.org/10.5281/zenodo.15412178.
[29] Sabato Nocera, Simone Romano, Massimiliano Di Penta, Rita Francese, Giuseppe Scanniello, On the adoption of software bill of materials in open-source software projects, J. Syst. Softw. (2025) 112540, http://dx.doi.org/10.1016/j.jss.2025.112540, URL.
[30] GitHub, Help improve GitHub dependency graph with your feedback!, 2023, URL https://github.com/orgs/community/discussions/43364.
[31] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, Experimentation in Software Engineering, Springer, 2012.